

Symbiosis Institute of Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE • JAVASCRIPT PROGRAMMING LAB

Student: Tanmay Sankulwar

PRN: 24070521058

Course: Semester V (B.Tech)

Experiment No. 5

Student Name: Tanmay Sankulwar
PRN: 24070521058
File Path: EXP5/index.html , EXP5/script.js , EXP5/style.css , EXP5/Case study/

Experiment Title

Shopping Cart Calculator with Advanced Array Operations

Software / Tools Required

1. Visual Studio Code / Antigravity IDE
2. Google Chrome (or modern web browser)
3. HTML5
4. CSS3
5. JavaScript (ES6)

Experiment Program Code

Task 5.a — Shopping Cart Interface and Product Collection

File: EXP5/index.html

The HTML document provides an interface to enter product names, unit prices, and quantities, dynamic table rendering for items in the cart, discount breakdown, item summary, and high-value product filtering.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Shopping Cart Calculator</title>
  <style>
    body {
      font-family: Arial;
      background: #f2f2f2;
    }
    .container {
      width: 800px;
      margin: 30px auto;
      background: white;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }
    input {
```

```

        padding: 8px;
        width: 150px;
        margin: 5px;
    }
    button {
        padding: 10px 15px;
        cursor: pointer;
    }
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }
    table, th, td {
        border: 1px solid black;
    }
    th {
        background: #007BFF;
        color: white;
    }
    th, td {
        padding: 10px;
        text-align: center;
    }
    .result {
        margin-top: 20px;
        font-size: 18px;
    }
}
</style>
</head>
<body>

<div class="container">
    <h2>Shopping Cart Calculator</h2>

    <input type="text" id="name" placeholder="Product Name">
    <input type="number" id="price" placeholder="Price">
    <input type="number" id="qty" placeholder="Quantity">

    <button onclick="addProduct()">Add Product</button>

    <table id="cartTable">
        <tr>
            <th>ID</th>
            <th>Product</th>
            <th>Price</th>
            <th>Quantity</th>
            <th>Total</th>
        </tr>
    </table>

    <div class="result" id="result"></div>

    <h3>Item Summary</h3>
    <ul id="summary"></ul>

    <h3>Expensive Products (Price > ₹1000)</h3>
    <ul id="expensive"></ul>
</div>

<script src="script.js"></script>
</body>
</html>

```

Task 5.b — Array Operations with `forEach()`, `reduce()`, `map()`, and `filter()`

File: EXP5/script.js

This script manages the shopping cart using an array of objects. It illustrates key Higher-Order Array Methods in JavaScript: - `cart.push()` to append new item objects. - `cart.forEach()` to iterate over items and generate table rows. - `cart.reduce()` to aggregate line-item totals and compute tiered discounts. - `cart.map()` to transform items into summary list items. - `cart.filter()` to extract products priced over ₹1000.

```
// Array to store cart products
let cart = [];

function addProduct() {
  let name = document.getElementById("name").value;
  let price = parseFloat(document.getElementById("price").value);
  let qty = parseInt(document.getElementById("qty").value);

  // Input validation
  if (name === "" || isNaN(price) || isNaN(qty)) {
    alert("Please enter all fields");
    return;
  }

  // Create product object
  let product = {
    id: cart.length + 1,
    name: name,
    price: price,
    quantity: qty
  };

  // Add product to cart
  cart.push(product);

  // Display cart
  displayCart();

  // Clear input fields
  document.getElementById("name").value = "";
  document.getElementById("price").value = "";
  document.getElementById("qty").value = "";
}

function displayCart() {
  let table = document.getElementById("cartTable");

  // Display table headings
  table.innerHTML = `
    <tr>
      <th>ID</th>
      <th>Product</th>
      <th>Price</th>
      <th>Quantity</th>
      <th>Total</th>
    </tr>
  `;

  // Display products using forEach()
  cart.forEach(function(item) {
    table.innerHTML += `
      <tr>
        <td>${item.id}</td>
        <td>${item.name}</td>
        <td>₹${item.price}</td>
        <td>${item.quantity}</td>
        <td>₹${item.price * item.quantity}</td>
      </tr>
    `;
  });

  // reduce() - Calculate total amount
  let total = cart.reduce(function(sum, item) {
    return sum + (item.price * item.quantity);
  });
}
```

```
    }, 0);

    // Discount Logic
    let discount = 0;
    if (total >= 50000) {
        discount = total * 0.20;
    }
    else if (total >= 20000) {
        discount = total * 0.10;
    }
    else if (total >= 5000) {
        discount = total * 0.05;
    }

    // Calculate final amount
    let finalAmount = total - discount;

    // Display result
    document.getElementById("result").innerHTML = `
        <b>Total Amount:</b> ₹${total.toFixed(2)}<br>
        <b>Discount:</b> ₹${discount.toFixed(2)}<br>
        <b>Final Amount:</b> ₹${finalAmount.toFixed(2)}
    `;

    // map() - Create item summary
    let summary = document.getElementById("summary");
    summary.innerHTML = "";

    cart.map(function(item) {
        summary.innerHTML += `
            <li>${item.name} : ₹${item.price * item.quantity}</li>
        `;
    });

    // filter() - Find expensive products
    let expensive = document.getElementById("expensive");
    expensive.innerHTML = "";

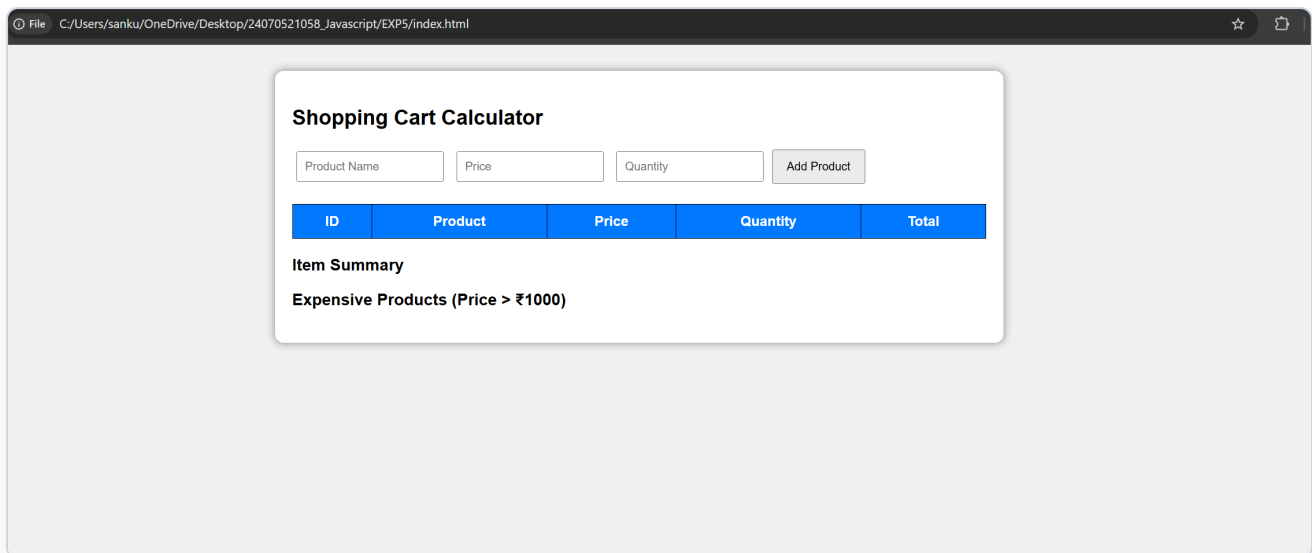
    let exp = cart.filter(function(item) {
        return item.price > 1000;
    });

    // Display expensive products
    exp.forEach(function(item) {
        expensive.innerHTML += `
            <li>${item.name}</li>
        `;
    });
}
```

Output

1. **Adding Products:** Every product added appends a new record to the internal array and renders an updated row into the cart table.
2. **Aggregated Bill Calculation (`reduce`):** Total amount is computed across all objects. A progressive discount rule applies (5% for \$≥ 5000\$, 10% for \$≥ 20000\$, 20% for \$≥ 50000\$).
3. **Item Summary (`map`):** An itemized list of product titles and calculated line totals is produced.
4. **Filtering High-Cost Items (`filter`):** Any item with a unit price \$> 1000\$ is isolated and displayed in the Expensive Products section.

Screenshot

*Shopping Cart Calculator Output*

Case Study

Case Study — Student Marks Management System

File: EXP5/Case study/index.html , EXP5/Case study/script.js , EXP5/Case study/style.css

The case study provides a complete practical implementation of fundamental JavaScript array operations on an array of student objects `{ name: string, marks: number }`. It offers visual before-and-after JSON snapshots and statistical calculations.

Methods Implemented in Case Study:

- `push()` : Appends a student record to the end of the array.
- `pop()` : Removes the last student record from the array.
- `unshift()` : Inserts a student record at index 0.
- `shift()` : Removes the first student record from the array.
- `splice(start, deleteCount, ...items)` : Deletes or replaces records at arbitrary indices.
- `slice(start, end)` : Extracts a shallow copy of a portion of the array without mutating original state.
- `map()` : Extracts mark lists and awards +5 grace marks (capped at 100).
- `filter()` : Selects students who scored ≥ 40 (passing mark).
- `reduce()` : Computes total cumulative marks.
- `Math.max(...marks)` & `Math.min(...marks)` : Determines highest and lowest scores using spread syntax.

```
// Sample snippet from EXP5/Case study/script.js
// Calculate results using modern array methods
document.getElementById('calcBtn').addEventListener('click', () => {
  const passMark = parseFloat(document.getElementById('passMark').value) || 40;

  // Extract marks array using map()
  const marksArray = students.map(s => s.marks);

  // MAX and MIN using Math methods and spread operator
  const max = marksArray.length ? Math.max(...marksArray) : 0;
  const min = marksArray.length ? Math.min(...marksArray) : 0;
```

```
// REDUCE to calculate total marks
const total = marksArray.reduce((acc, m) => acc + m, 0);
const avg = marksArray.length ? (total / marksArray.length) : 0;

// FILTER for passing students
const passed = students.filter(s => s.marks >= passMark);

// MAP for awarding grace marks
const grace = students.map(s => ({ name: s.name, marks: Math.min(100, s.marks + 5) }));

// Update statistical UI
document.getElementById('totalStudents').textContent = students.length;
document.getElementById('totalMarks').textContent = total;
document.getElementById('averageMarks').textContent = avg.toFixed(2);
document.getElementById('maxMarks').textContent = max;
document.getElementById('minMarks').textContent = min;
});
```

Case Study Screenshot

The screenshot shows a web application titled "Student Marks Management System". It features an "Add Student" form with fields for "Student Name" and "Marks (0-100)", and an "Add Student" button. Below this is a "Students" table with columns: Sr. No., Student Name, Marks, and Grade. The "Results" section displays five summary statistics: Total Students (0), Total Marks (0), Average (0), Maximum (0), and Minimum (0). There is a text input field with the value "40" and a "Calculate Results" button. At the bottom, a row of statistics is shown: "Students who passed", "Marks after map() (+5 grace, max 100)", "Total using reduce()", and "forEach() display", with the value "0" displayed under "Total using reduce()". The footer text reads "Demo: JavaScript array methods for college practical."

Student Marks Management System

Result / Conclusion

Experiment 5 provided in-depth practical mastery over JavaScript Array manipulation techniques. Both core and case study implementations validated the usage of mutating array functions (`push` , `pop` , `shift` , `unshift` , `splice`) and non-mutating functional programming methods (`map` , `filter` , `reduce` , `slice`).